PCC104 - Projeto e Análise de Algoritmos

Prof. Rodrigo Silva

20 de março de 2026

UNIVERSIDADE FEDERAL DE OURO PRETO

Outline

Algoritmo

Introdução à Análise de Algoritmos

Algoritmo

- Procedimento computacional
- Bem definido
- Toma algum valor ou conjunto de valores de entrada
- Produz algum valor ou conjunto de valores de saída

Procedimento Computacional

- Sequência de passos bem definidos (não ambíguos)
- Cada passo descreve uma operação elementar
- Cada passo é simples e executável em tempo finito

Algoritmo

- Procedimento para resolver um problema computacional

Problema Computacional

Definição

Um problema computacional é a especificação de uma tarefa que define, para cada valor de entrada, quais valores de saída são considerados corretos.

Exemplo: Ordenação

Entrada: uma sequência de números

$$(a_1, a_2, \ldots, a_n)$$

Saída: uma permutação da sequência tal que

$$a_1 \leq a_2 \leq \cdots \leq a_n$$

Instância de um Problema

Definição

Uma instância de um problema é um conjunto específico de valores de entrada para um problema computacional.

Exemplo: Problema de Ordenação

Problema: ordenar uma sequência de números.
Instância:

$$(8, 3, 5, 2, 9)$$

Problema, Instância e Algoritmo

Conceitos

- **Problema** $\rightarrow$ classe de tarefas
- **Instância** $\rightarrow$ caso específico
- **Algoritmo** $\rightarrow$ método para resolver

Exemplo

Problema: ordenação

$$\Downarrow$$

Instância: $(8, 3, 5, 2, 9)$

$$\Downarrow$$

Algoritmo

$$\Downarrow$$

Saída: $(2, 3, 5, 8, 9)$

Algoritmo Correto

Definição

Um algoritmo é dito correto se, para toda instância válida do problema, ele produz uma saída que satisfaz a especificação do problema em tempo finito.

Ordenação por Inserção (Insertion Sort)

Problema de Ordenação

Entrada: Uma sequência de n números

$$\langle a_1, a_2, \ldots, a_n \rangle$$

Saída: Uma permutação (reordenação) da sequência de entrada

$$\langle a'_1, a'_2, \ldots, a'_n \rangle$$

tal que

$$a'_1 \leq a'_2 \leq \cdots \leq a'_n$$

Objetivo

Reordenar os elementos da entrada em ordem não decrescente.

Ordenação por Inserção

Insertion-Sort(A)

```
for j ← 2 to A.length do
    key ← A[j]
                ▷ Insere A[j] na sequência ordenada A[1 ... j − 1]
    i ← j − 1
    while i > 0 and A[i] > key do
        A[i + 1] ← A[i]
        i ← i − 1
    end while
    A[i + 1] ← key
end for
```

Exemplo : $[5, 2, 4, 6, 1, 3]$

Corretude da ordenação por inserção

Invariante de Laço

Uma **invariante de laço** é uma propriedade que deve ser verdadeira:

- **Inicialização:** antes da primeira iteração do laço;
- **Manutenção:** antes de cada iteração, permanece verdadeira após a execução do corpo do laço;
- **Término:** ao final do laço, fornece uma propriedade útil para demonstrar a correção do algoritmo.

Uso

Invariantes de laço são utilizadas para demonstrar a **correção** de algoritmos iterativos.

Correção do Insertion Sort

Invariante de Laço

No início de cada iteração do laço **for** (linha 1), o subarray

$$A[1 \ldots j-1]$$

consiste nos elementos originalmente em $A[1 \ldots j-1]$, porém em ordem não decrescente.

Prova

Inicialização: Para $j = 2$, o subarray $A[1]$ contém apenas um elemento, logo está ordenado.

Manutenção: A cada iteração, o elemento $A[j]$ é inserido na posição correta em $A[1 \ldots j-1]$, produzindo $A[1 \ldots j]$ ordenado.

Término: Quando o laço termina ($j = A.\text{length} + 1$), temos

$$A[1 \ldots A.\text{length}]$$

ordenado.

O que é Análise de Algoritmos?

Definição

A **análise de algoritmos** consiste em prever os recursos computacionais necessários para executar um algoritmo, tipicamente em função do tamanho da entrada n.

Por que analisar algoritmos?

- Comparar diferentes algoritmos para o mesmo problema;
- Prever desempenho para entradas grandes;
- Compreender como o tempo de execução cresce com n;
- Guiar o projeto de algoritmos mais eficientes.

Desafio

Para analisar um algoritmo, precisamos de um modelo que defina:

- O que é uma operação;
- Quanto custa cada operação.

Modelo de Computação (RAM -*Random-access Machine*)

Na análise de algoritmos, assumimos o modelo RAM, no qual:

- As instruções são executadas sequencialmente;
- Cada operação básica (atribuição, soma, comparação, acesso a memória) custa tempo constante;
- O acesso à memória é uniforme (tempo constante para qualquer posição).

Cuidados

- O custo constante pode variar entre operações (ex.: divisão vs. soma);
- Ignoramos efeitos de hardware (cache, paralelismo, I/O);
- Assumimos que os dados cabem na memória principal.

Limitações

- Não modela hierarquia de memória;
- Não captura custo de operações com números muito grandes;
- Pode não refletir desempenho real em sistemas modernos.

Análise do Insertion Sort

Por que analisar?

- Determinar o custo computacional do algoritmo
- Comparar diferentes algoritmos para o mesmo problema
- Prever comportamento em diferentes tamanhos de entrada

Modelo de Análise (CLRS)

- Modelo **RAM** (Random Access Machine)
- Cada operação elementar tem custo constante
- Tempo total = número de execuções de cada linha

Ideia central

- O tempo depende do número de deslocamentos realizados
- Esse número varia conforme a **ordem da entrada**

Especificando a Análise

- Modelo RAM
- Tamanho da entrada?

Insertion Sort: Custo e Frequência

Ordena-Por-Inserção(A, n)

1. **for** $i = 2$ **to** n
2. $\quad$ chave $= A[i]$
3. $\quad$ $j = i - 1$
4. $\quad$ **while** $j > 0$ **and** $A[j] >$ chave
5. $\quad\quad$ $A[j+1] = A[j]$
6. $\quad\quad$ $j = j - 1$
7. $\quad$ $A[j+1] =$ chave

Custo: n^o Vezes

- $c_1 : n - 1$
- $c_2 : n - 1$
- $c_3 : n - 1$
- $c_4 : n - 1$
- $c_5 : \sum_{i=2}^{n}(t_i - 1)$
- $c_6 : \sum_{i=2}^{n}(t_i - 1)$
- $c_7 : n - 1$

Observação

t_i é o número de vezes que o teste do `while` é executado para cada i.

Análise Insertion Sort

Tempo Total

$$T(n) = c_1(n-1) + c_2(n-1) + c_3(n-1) + c_4(n-1)$$
$$+ c_5 \sum_{i=2}^{n}(t_i - 1) + c_6 \sum_{i=2}^{n}(t_i - 1) + c_7(n-1)$$

Observação

Mesmo para entradas de mesmo tamanho o tempo de execução pode sdepender de qual entrada desse tamanho é dada.

Análise Insertion Sort - Entrada ordenada

Tempo Total

$$T(n) = c_1(n-1) + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_5\sum_{i=2}^{n}(t_i - 1) + c_6\sum_{i=2}^{n}(t_i - 1) + c_7(n-1)$$

$T(n) = an + b$ (Tempo linear)

Análise Insertion Sort - Ordem Decrescente

Tempo Total

$$T(n) = c_1(n-1) + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_5\sum_{i=2}^{n}(t_i - 1) + c_6\sum_{i=2}^{n}(t_i - 1) + c_7(n-1)$$

$T(n) = an^2 + bn + c$ (Tempo quadrático)

Casos de Análise de Algoritmos

Melhor Caso

- Menor tempo de execução para entradas de tamanho n
- Ocorre quando a entrada favorece o algoritmo
- Formalmente:

$$T_{\text{best}}(n) = \min_{\text{entradas de tamanho } n} T(n)$$

Casos de Análise de Algoritmos

Pior Caso

- Maior tempo de execução para entradas de tamanho n
- Garante limite superior de desempenho
- Formalmente:

$$T_{\text{worst}}(n) = \max_{\text{entradas de tamanho } n} T(n)$$

Casos de Análise de Algoritmos

Caso Médio

- Tempo esperado sobre todas as entradas de tamanho n
- Depende de uma distribuição de probabilidade
- Formalmente:

$$T_{\text{avg}}(n) = \mathbb{E}[T(n)]$$

Por que analisar o pior caso?

Garantia de Desempenho

- Fornece um limite superior para o tempo de execução
- Garante que o algoritmo nunca será pior do que isso
- O caso médio é quase tão ruim quanto o pior caso

Independência da Entrada

- Não depende de hipóteses sobre a distribuição dos dados
- Evita suposições difíceis de justificar na prática

Robustez

- Essencial em sistemas críticos (tempo real, segurança, etc.)
- Protege contra entradas adversariais

Caracterização dos Tempos de Execução